



Sign In

Get Started

BLOG ▾

Category ▾

[Home](#) > [About R](#) > [Learn R](#)

Importing Data Into R - Part Two

A tutorial on importing data into R. The focus is on reading data from sources like statistical software, databases, webscraping, and more.

Mar 2019 · 34 min read

**Karlijn Willems**

Former Data Journalist at DataCamp | Manager at NextWave Consulting

TOPICS

[R Programming](#)[Data Science](#)

In this follow-up tutorial of [This R Data Import Tutorial Is Everything You Need-Part One](#), DataCamp continues with its comprehensive, yet easy tutorial to quickly import data into R, going from simple, flat text files to the more advanced SPSS and SAS files.

As a lot of our readers noticed correctly from the first post, some great packages to import data into R haven't yet received any attention, nor did the post cover explicitly the distinction between working with standard data sets and large data sets. That is why this will be the focus of today's post.

Keep on reading to discover other and new ways to import your specific file into R, and feel free to [reach out](#) if you have additional questions or spot an error we should correct.

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)**Accept**

Next, the data from other sources like statistical software, databases, web scraping, etc. will be discussed.

If you want to know more about the possible steps that you might need to undertake before importing your data, go to our [first post](#), which explains how you can prepare your data and workspace before getting your data into R.

Reading in Flat Files Into R with `scan()`

Besides `read.table()`, which was mentioned in the [first post of the R data import tutorial](#), the `scan()` function can also work when handling data that is stored in simple delimited text files. Unlike the `read.table()` function, the `scan()` function returns a list or a vector, not a **dataframe**.

Suppose you have the following `.txt` document:

```
24 1991
21 1993
53 1962
```



POWERED BY DATACAMP WORKSPACE

You can read in the data (which you can download [here](#)) with the following command:

```
data <- scan("birth.txt")
```



POWERED BY DATACAMP WORKSPACE

Note that your file can also be an online data set. In that case, you just pass the URL as the first argument of the `scan()` function.

Alternatively, you could also read in the data into a matrix:

```
data <- matrix(scan("birth.txt"),
               nrow=2,
               byrow=TRUE)
```



DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

```
data <- scan("age.txt",  
            what = list(Age = 0,  
                        Birthyear= 0),  
            skip=1,  
            quiet=TRUE)
```



POWERED BY DATACAMP WORKSPACE

Note how you first pass the (path to the) file with its extension as an argument (depending on whether you set the working directory to the folder that holds your dataset or not) and then specify the type of data to be read in, whether or not you want to skip the first line of your dataset, which character delimits the fields and if you want to print a line that says how many items have been read.

If your data can also contain other data types, you should tweak the `scan()` function a little bit, just like in this example:

```
data <- scan("age.csv",  
            what = list(Age = 0,  
                        Name= "",  
                        Birthyear= 0),  
            skip=1,  
            sep=";",  
            quiet=TRUE)
```



POWERED BY DATACAMP WORKSPACE

Tip: you can do this yourself, too! Download the text file that was used above [here](#).

And then you can also read in the data in a data frame:

```
data <- data.frame(scan("age.csv",  
                       what = list(Age = 0,  
                                   Name = "",
```



DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

you can use with the `read.table()` function. That is why it's always handy to check out the documentation! Go [here](#) if you want to read up on the `scan()` function's arguments.

Remember that you can get the working directory and set it with the following commands, respectively:

```
getwd()
setwd("<path to your folder>")
```



POWERED BY DATACAMP WORKSPACE

Getting Fixed Column Data Into R with `read.fwf()`

To read a table of “fixed width formatted data” into a data frame in R, you can use the `read.fwf()` function from the `utils` package.

You use this function when your data file has columns containing spaces, or columns with no spaces to separate them.

```
Phys / 00 / 1:    M  abadda
Math / 00 / 2:    F  bcdccb
Lang / 00 / 3:    F  abcdab
Chem / 00 / 4:    M  cdabaa
```



POWERED BY DATACAMP WORKSPACE

Here, you do know that, for example, the subject values always reside in the first 7 characters of each line and the sex values are always at 22, and the scores start from character 25 to 30.

If you want to try out loading these data into R, you can easily download the text file [here](#).

You would need to execute the following command to get the data from above correctly into R:

```
read.fwf("scores.txt",
         widths= c(7,-14,1,-2,1,1,1,1,1,1),
```



DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

represent the sex, but you don't want the two following characters, so you pass `-2`. All following characters need to be read in into separate columns, so you split them by passing `1,1,1,1,1` to the argument. Of course, these values can and will differ, depending on what columns you want to import.

There are a number of extra arguments that you can pass to the `read.fwf()` function. Click [here](#) to read up on them.

Note that if you want to load in a file using Fortran-style format specifications, you can use the `read.fortran()` function:

```
data <- tempfile()  
cat(file = data, "345678", "654321", sep = "\n")  
read.fortran(data, c("F2.1", "F2.0", "I2"))
```



POWERED BY DATACAMP WORKSPACE

As you can see from the small example above, you use Fortran-style format specifications as a second argument to the `read.fortran()` function. The arguments that you could possibly pass are in the style of: `"rFl.d"`, `"rDl.d"`, `"rXl"`, `"rAl"` or `"rll"`, where `"l"` is the number of columns, `"d"` is the number of decimal places, and `"r"` is the number of repeats. In this case, you see `2.1`, `2.0` and `2` listed by means of the `c()` function, which means that you have three columns with two rows. In the first column, you have values with one decimal place, in the second and the third also contain values with no decimal place.

For what concerns the type of values that the columns contain, you can have:

- `"F"` and `"D"` for numeric formats;
- `"A"` if you have character values;
- `"I"` for integer values;
- And `"X"` to indicate columns that can be skipped.

In this case, the first and second columns will contain numeric formats, while the third column contains integer values.

Note that the repeat code `"r"` and decimal place code `"d"` are always optional. The length code `"l"` is required except for `"X"` formats when `"r"` is present.

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

Apart from the `xlsx` package, you also have a number of other options to read spreadsheets into R:

1. Reading Excel Spreadsheets into R From The Clipboard

If you have a spreadsheet open, you can actually copy the contents to your clipboard and import them quickly into R. To do this, you can either use the `readClipboard()` or `read.table()` functions:

```
readClipboard() #Only on Windows
read.table(file="clipboard")`
```



POWERED BY DATACAMP WORKSPACE

As you will see if you try this out, the first approach works well for vector data, but it gets pretty complicated if you have tabular data in your clipboard. If you want to know more about `read.table()`, you should definitely go to the [first part](#) of the R data import tutorial or our tutorial on [reading and importing Excel files into R](#).

2. Reading Excel Spreadsheets into R Through The RODBC Package

The second way to get your Excel spreadsheets into R is through the `RODBC` package:

- A first way to use this package is like this:

```
library(RODBC)
connection <- odbcConnect("<DSN>")
```



POWERED BY DATACAMP WORKSPACE

Note that the argument that you pass to `odbcConnect()` is actually a DSN. For a complete guide on how to set up your DSN, on how to set up a connection, etc., go to [this page](#) an extensive, yet easily accessible tutorial!

- Once you have set up your connection, you could also use the `sqlQuery()` function to get data from `.xls` spreadsheets:

```
query <- "SQL Query"
```



DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

At the end of an R session, don't forget to close the connections:

```
odbcCloseAll()
```



POWERED BY DATACAMP WORKSPACE

Tip: If you want to know more about importing spreadsheets or Excel files into R, definitely go to our first tutorial on [importing data into R](#) or consider reading our tutorial on [reading and importing Excel files into R](#), which deals with the [readxl](#) and [XLConnect](#) packages, among others.

Importing Google Spreadsheets into R

The [googlesheets](#) package with its [gs_read\(\)](#) function allows you to read in Google spreadsheets into R.

Start by executing the following line of code:

```
gs_ls()
```



POWERED BY DATACAMP WORKSPACE

Let the browser start up and complete the authentication process. Then, if you want to read in the data or edit it, you have to register it. You can do this by specifying your spreadsheet by title or by key:

```
data <- gs_title("<your spreadsheet>")  
data <- gs_key()
```



POWERED BY DATACAMP WORKSPACE

Next, you can read in the data:

```
gs_read(data)
```



DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

```
data <- read.DIF("<your spreadsheet>",  
                header=FALSE,  
                as.is = !stringsAsFactors)
```



POWERED BY DATACAMP WORKSPACE

Note that you can specify whether your spreadsheet has a header or not and whether you want to import the data “as is”, that is, whether you want to convert character variables to convert to factors. In this case, you didn't want to have this, so you gave in `!stringsAsFactors` .

For more information on this function or its arguments, go to [this page](#).

Getting Excel Files into R

Besides spreadsheets, you might also be interested in getting your actual Excel files into R. Look no further and keep on reading to find out how you can do this!

Note that this post only elaborates on what has been described in our tutorial on [reading and importing Excel files into R](#) and our first [This R Data Import Tutorial Is Everything You Need](#) post!

Importing Excel Files into R With `readxl`

Even though this package is still [under active development](#), it's really worth to check it out, because it offers you a pretty easy way to read in Excel files:

```
library(readxl)  
read_excel("<path to file")
```



POWERED BY DATACAMP WORKSPACE

Remember that you can just type the file's name, together with its extension if your folder is in your working directory. Get and set your working directory through the following lines of code:

```
## Get working directory
```

```
## Set working directory
```

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

Reading in Excel Files into R With `openxlsx`

The `openxlsx` package also provides you with a simple way to read Excel `.xlsx` files into R:

```
library(openxlsx)
read.xlsx("<path to your file>")
```



POWERED BY DATACAMP WORKSPACE

If you want to know more details on this package or on the arguments that you can pass to the `read.xlsx()` function, definitely click [here](#).

Tip: If you want to know more about importing Excel files into R, definitely go to our first tutorial on [“importing data into R”](#) or consider reading our extensive tutorial on [reading and importing Excel files into R](#), which also deals with the `XLConnect` package, amongst others.

Getting OpenDocument Spreadsheets into R

Use the `read.ods()` function from the `readODS` package to read in your OpenDocument spreadsheets into R and put them into data frames:

```
library(readODS)
read.ods("<path to your file>",
        sheet = 1,
        formulaAsFormula = FALSE)
```



POWERED BY DATACAMP WORKSPACE

Note that, apart from the file that you want to get into R, you can specify the sheet that you want and that you have the possibility to display formulas as formulas (for example, “SUM(B1:B3)” or the resulting values).

Importing JavaScript Object Notation (JSON) Files into R

In our [first post](#) on importing data into R, the `rjson` package was mentioned to get **JSON**

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

You import JSON files with the `fromJSON()` function:

```
library(jsonlite)
data <- fromJSON("<Path to your JSON file>")
```



POWERED BY DATACAMP WORKSPACE

For a well-explained quickstart with the `jsonlite` package, go [here](#).

Importing JSON Files into R with the `RJSONIO` package

The third, well-known package to get JSON files into R is `RJSONIO`. Just like the `jsonlite` and the `jsonlite` packages, you use the `fromJSON()` function:

```
library(RJSONIO)
data <- fromJSON("<Path to your JSON file")
```



POWERED BY DATACAMP WORKSPACE

The Best JSON Package?

There has been considerable discussion about this topic. If you want to know more, you should definitely check out the following pages and posts:

- [this page](#) offers mainly illustrations through code examples which provide you with more insight into the behavior and performance of **JSON** packages in R.
- Definitely read [this blog post](#), which tries to figure out which package handles JSON data best in R.

Getting Data from Statistical Software Packages into R

If your data is not really spreadsheet-like and isn't an Excel or JSON file, it might just be one that is made with one of the many statistical software packages.

This section will provide you with more ways to read in your SPSS, Stata or SAS files, while also giving an overview of importing files that come from S-plus and Epi Info. Definitely

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

The **haven** package offers the **read_spss()** function to read SPSS files into R:

```
library(haven)
data <- read_spss("<path to your SPSS file>")
```



POWERED BY DATACAMP WORKSPACE

Importing Stata Files into R

Similar to the **foreign** package, the **haven** package also provides a function to read in Stata files into R, namely **read_dta()**:

```
data <- read_dta("<path to your STATA file>")
```



POWERED BY DATACAMP WORKSPACE

Remember always to install your packages if necessary and to activate them in your workspace. For example, you can install and activate the **haven** package in your workspace with the following commands:

```
install.packages("haven")
library(haven)
```



POWERED BY DATACAMP WORKSPACE

Importing SAS Files into R

Since the **sas7bdat** package was cited in the last post, this follow-up tutorial will focus on other ways to read in SAS files:

1. How To Import SAS XPORT Files into R With The **foreign** package

The **foreign** package with the **read.xport()** function also allows you to get your SAS XPORT files into R:

```
library(foreign)
```



DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

```
library(SASxport)
data <- read.xport("<path to your SAS file>")
```



POWERED BY DATACAMP WORKSPACE

3. How To Import SAS Files into R With The `haven` Package

Just like the `foreign` and the `sas7bdat` packages, the `haven` package also allows you to read in b7dat files into R with the `read_sas()` function:

```
library(haven)
data <- read_sas("<path to your SAS file>")
```



POWERED BY DATACAMP WORKSPACE

Getting S-plus Files Into R

For old S-plus datasets, namely those that were produced on either Windows versions 3.x, 4.x or 2000 or Unix, version 3.x with 4 byte integers, you can use the `read.S()` function from the `foreign` package:

```
library(foreign)
data <- read.S("<Path to your file>")
```



POWERED BY DATACAMP WORKSPACE

Reading In Epi Info Files Into R

As you may have read in our previous tutorial or in this one, the `foreign` package offers many functions to read in specific files into R, and Epi Info is one of them. You can just use the `read.epiinfo()` function to get your data into R:

```
library(foreign)
data <- read.epiinfo("<Path to your file>")
```



DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

A few are listed below. Keep on reading!

Importing MATLAB Files into R

You can use the `R.matlab` package with its `readMat()` function to import **MATLAB** files into R.

You can either pass a character string as a first argument to this function or you can pass a raw vector. In the first case, your input would be interpreted as a filename, while in the second case it will be considered a raw binary connection:

```
library(R.matlab)
data <- readMat("<Path to your file>")
```



POWERED BY DATACAMP WORKSPACE

The `readMat()` function will return a named list structure that contains all variables from the MAT file that you imported.

Reading In Octave Files into R

The `foreign` package is here again! Use the `read.octave()` function to import **Octave** text data into R:

```
library(foreign)
data <- read.octave("<Path to your file>")
```



POWERED BY DATACAMP WORKSPACE

Getting FitbitScraper Data into R

You can use the `fitbitScraper` package to get data from **fitbit**.

(For those who aren't familiar with the company: the company offers products such as activity trackers and other technology devices that measure personal data such as the number of steps walked or the quality of sleep.)

Go [here](#) for a short and practical tutorial on how you can use the `fitbitScraper` package.

Importing Quantmod Data into R

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

Note that first you specify a character vector with the names of each symbol to be loaded. In this case, that is "YHOO". Then, you define a sourcing method. The sourcing methods that are available at this point in time are yahoo, google, MySQL, FRED, csv, RData, and oanda.

Next, you specify the lookup parameters and save them for future sessions:

```
setSymbolLookup(YHOO='google',GOOG='yahoo')  
saveSymbolLookup(file="mysymbols.rda")
```



POWERED BY DATACAMP WORKSPACE

The new sessions then call

```
loadSymbolLookup(file="mysymbols.rda")  
getSymbols(c("YHOO","GOOG"))
```



POWERED BY DATACAMP WORKSPACE

If you want more information on quantitative finance applications in R, click [here](#) or go to [this page](#) for a detailed tutorial for beginners on working with quantmod.

Getting ARFF Files into R

Data from [Weka](#) Attribute-Relation File Format (ARFF) files can be read in with the `read.arff()` function:

```
library(foreign)  
data <- read.arff("<Path to your file>")
```



POWERED BY DATACAMP WORKSPACE

For more information on this function, go to [this page](#).

Note that the `RWeka` package also offers the same function to import ARFF files. Go [here](#) if you want to know more!

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

- For help with [RMongo](#) , you can go to [this blog post](#);
- The [RODBC](#) package is extensively discussed also in [this tutorial](#). For a shorter description of the working method, go [here](#);
- To learn how to work with [ROracle](#) , you can check out [this presentation](#);
- For more information on how to work with [RPostgreSQL](#) , go [here](#) or [here](#);
- Definitely go to [this very informative tutorial](#) if you want to know more about working with [SQLite](#) ;
- Lastly, to connect to SQL Server with [RJDBC](#) , you should check out [this blog post](#) or go to [this page](#) if you want to connect to an Oracle database. **Note** that this last page also gives you other options to connect to Oracle databases.

Note that there is also a database interface package [DBI](#) which allows communication between R and relational database management systems. For more information, click [here](#).

Some explanations on how to work with this package can be found [here](#).

Getting Binary Files into R

Binary data files contain information that is stored in groups of binary digits. Each binary digit is a zero or one. Eight binary digits that are grouped together form a byte. You can read in binary data with the [readBin\(\)](#) function:

```
connection <- file("<path to your file>", "rb") #You open the connection
data <- readBin(connection,
                  what="numeric") #Mode of the vector to be read
```

POWERED BY DATACAMP WORKSPACE

For a more detailed example, go to [this page](#). For more information on the `readBin()` function, click [here](#).

Reading In Binary Data Formats into R

The packages [hdf5](#) , [h5r](#) , [rhdf5](#) , [RNetCDF](#) , [ncdf](#) and [ncdf4](#) provide interfaces to NASA's [HDF5](#) and to UCAR's [netCDF](#) data files.

For those of you who are interested in some tutorials on how to work with HDF5 or netCDF

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

A DBF or DataBase File is the underlying format of **dBase**. You can read in DBF files with the use of the **foreign** package, which offers the **read.dbf()** function:

```
library(foreign)
data <- read.dbf("<Path to your file>")
```



POWERED BY DATACAMP WORKSPACE

Note that if you're using Windows, you can also use the **RODBC** package with the **odbcConnectDbase()** function to read DBF files via Microsoft's dBase ODBC driver.

Importing Flat Contingency Tables into R

The **foreign** package allows you to read multiple file formats; 'Flat' contingency tables are no exception. You can use the **read.ftable()** function to accomplish this:

```
library(foreign)
data <- read.ftable("<Path to your file>")
```



POWERED BY DATACAMP WORKSPACE

Remember that “flat” contingency tables are very similar to the “normal” contingency tables: they contain the counts of each combination of the levels of the variables (factors) involved. However, this information is re-arranged as a matrix whose rows and columns correspond to the unique combinations of the levels of the row and column variables. “Flat” contingency tables are therefore often preferred to represent higher-dimensional contingency tables.

Reading in Geographical Information System (GIS) Files into R

You can use the **rgdal** and **raster** packages, amongst others, to get your GIS files into R.

If you're not sure how to start using the **rgdal** package, consider checking out [this nice blog post](#), which introduces you to working with geospatial data in R.

You can also check out [this tutorial](#), which works with **rgdal** as well as with **raster**.

Importing Integrated Taxonomical Information (ITIS) Tables into R

You can import ITIS tables with the **read.ftable()** function.

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

Importing Large Data Sets into R

Importing large data sets often causes discussion amongst R users. Besides the packages that are meant to connect with databases, there are also some others that stand out when working with big data.

Importing Large Data Sets into R with the `data.table` Package

Described as the “fast and friendly file finagler”, the popular `data.table` package is an extremely useful and easy to use. Its `fread()` function is meant to import data from *regular* delimited files directly into R, without any detours or nonsense.

Note that “regular” in this case means that every row of your data needs to have the same number of columns. An example:

```
V1 V2 V3
1  1  6  a
2  2  7  b
3  3  8  c
4  4  9  d
5  5 10  e
```



POWERED BY DATACAMP WORKSPACE

One of the great things about this function is that all controls, expressed in arguments such as `sep`, `colClasses` and `nrows` are automatically detected. Also, `bit64::integer64` types are also detected and read directly without needing to read as character before converting.

Remember that `bit64::integer64` types are 64 bit integers: these numbers are stored in the computer as being 64 bits long. By default, these are 32 bits only. Because the `bit64::integer64` types are detected, the system knows it's a number and it's not being read in as a character to then be converted into an integer.

An example of the `fread()` function is:

```
library(data.table)
data <- fread("http://assets.datacamp.com/blog_assets/chol.txt")
. .
```



DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

```
##    3:  44    170     80  304  sigare    a  dead
##    4:  37    173     89  178  nonsmo    o  alive
##    5:  26    170     71  206  sigare    o  alive
##   ---
## 196:  35    174     57  222   pipe    a  alive
## 197:  38    172     91  227  nonsmo    b  alive
## 198:  26    170     60  167  sigare    a  alive
## 199:  39    165     74  259  sigare    o  alive
## 200:  49    178     81  275   pipe    b  alive
```

POWERED BY DATACAMP WORKSPACE

Note that reading in your data with the `fread()` function returns you a data table:

```
str(data)
```



POWERED BY DATACAMP WORKSPACE

```
## Classes 'data.table' and 'data.frame':  200 obs. of  7 variables:
## $ AGE      : int  20 53 44 37 26 41 39 28 33 39 ...
## $ HEIGHT: int  176 167 170 173 170 165 174 171 180 166 ...
## $ WEIGHT: int  77 56 80 89 71 62 75 68 100 74 ...
## $ CHOL    : int  195 250 304 178 206 284 232 152 209 150 ...
## $ SMOKE   : chr  "nonsmo" "sigare" "sigare" "nonsmo" ...
## $ BLOOD   : chr  "b" "o" "a" "o" ...
## $ MORT    : chr  "alive" "dead" "dead" "alive" ...
## - attr(*, ".internal.selfref")=<externalptr>
```

POWERED BY DATACAMP WORKSPACE

This is different from the `read.table()`, which creates a data frame of your data.

You can find more on the differences between data frames and data tables are explained [here](#). In short, the most important thing is to know that all data.tables are also data.frames:

data.table is a special data frame object. A data.table can be converted to a data.frame by using the `as.data.frame()` function.

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

```
na.strings = c("NA", "N/A", ""),
stringsAsFactors=FALSE
)
```

POWERED BY DATACAMP WORKSPACE

Note that the input may also be a file that you want to read in and doesn't always need to be a URL. Also, note how many of the arguments are the same as the ones that you use in `read.table()`, for example.

Tip: want to know more about `data.table`? Maybe our course on [Data Analysis In R, The data.table Way](#) can interest you! With the guidance of [Matt Dowle](#) and [Arun Srinivasan](#), you'll go from being a `data.table` novice to `data.table` expert in no time.

Getting Large Data Sets into R with the `ff` Package

The `ff` package allows for the “memory-efficient storage of large data on disk and fast access functions”. It's one of the solutions that frequently pops up when you're looking into discussions that deal with reading in big data as data frames, like [here](#).

If you want to import separated flat files into `ff` data frames, you can just use the `read.table.ffdf()`, `read.csv.ffdf()`, `read.csv2.ffdf()`, `read.delim.ffdf()` or `read.delim2.ffdf()` functions, much like the `read.table()` function and its variants or convenience wrappers, which are described [in one of our previous posts](#):

```
bigdata <- read.table.ffdf(file="<Path to file>",
                           nrows=n)
```



POWERED BY DATACAMP WORKSPACE

Note that your first argument can be `NULL` (like in this case) or can designate an optional `ffdf` object to which the read records are appended. If you want to know more, please go [here](#). Then, you name the file from which the data are read with the argument `file`. You can also specify a maximum number of rows to be read in with `nrows` (which is the same as you would do with `read.table()`!).

You can also go further and specify the file encoding, the levels or the name of a function that is called for reading each chunk:

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

Tip more arguments that you can add to the `read.table.ffdf()`, `read.csv.ffdf()`, `read.csv2.ffdf()`, `read.delim.ffdf()` or `read.delim2.ffdf()` functions can be found [here](#).

Importing Large Data Sets into R with `bigmemory`

Another package that frequently pops up in the search results for any query related to large data sets in R is the `bigmemory` package. This package allows you to “manage massive matrices with shared memory and memory-mapped files”.

Note that you can not use this package on Windows: there are [no Windows binaries available](#).

```
library(bigmemory)
bigdata <- read.big.matrix(filename="<File name>",
                           sep="",
                           header=TRUE,
                           skip=2)
```



As usual, you first give the file name to the function, and then you can begin to specify other things, like the separator symbol, the header or the number of lines to skip before starting to read in your file with the arguments `sep`, `header` and `skip` respectively.

Note that these are only a few examples! You can pass a lot more arguments to the `read.big.matrix()` function! Consider reading the [documentation](#) if you want to know more.

Reading in Large Data Sets into R with the `sqldf` Package

The `sqldf` package is also one of the packages that you might consider using when you're working with large data sets. This package allows you to “perform SQL selects on R”, and especially its `read.csv.sql()` function is very handy if you want to read a file into R, filtering it with an SQL statement. Then, only a portion of the data is processed by R:

```
library(sqldf)
bigdata <- read.csv.sql(file="<Path to your file>",
                        sql="select * from file where ...",
```



DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

are part of the file that you mention in the `file` argument of `read.csv.sql()`.

Tip: for more information on how to work with `sqldf`, you can go [here](#) for a video tutorial or [here](#) for a written overview of the basics.

Importing Large Data Sets into R with the `read.table()` Function

You can use the “standard” `read.table()` function to import your data, but this will probably take more time than other packages that are especially designed to work better with bigger data sets. To see how the `read.table()` function works, go back to our [first post](#).

To make this function go a little bit faster, you could tweak it yourself to get an optimized `read.table()` function. This tweaking actually only consists of adding arguments to the usual `read.table()` function, just like this:

```
df <- read.table("<Path to your file>",  
                header = FALSE,  
                sep=";",  
                quote = "",  
                na.strings = "EMPTY",  
                colClasses = c("character", "numeric", "factor"),  
                strip.white = TRUE,  
                comment.char="",  
                stringsAsFactors = FALSE,  
                nrow = n  
                )
```

POWERED BY DATACAMP WORKSPACE

Note that

- you first pass the (path to your) file, depending on whether you have set your working directory to the folder in which the file is located or not.
- Then, you use the `header` argument to indicate whether the file contains the names of the variables as its first line. This is not the case in the example above.
- The field separator character is set as `;` with the argument `sep`; This means that the values on each line of the file are separated by this character.

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

unquoted character fields; This is only applicable when you have used the `sep` argument!

- When `comment.char` is set as `""`, you turn off the interpretation of comments.
- You don't want characters to be converted to factors! That is why you have also defined `colClasses`. You confirm this by setting `stringsAsFactors` to `FALSE`.

Tip: this argument is, together with `colClasses` and `comment.char`, probably one of the more important ones if you want to import your data smoothly!

- Lastly, you put a maximum number of rows to read in.

Tip: if you want to have more information on all arguments that you can pass to the `read.table()` function, you should definitely consider reading our post on [reading Excel files into R](#).

Getting Large Data Sets into R with the `readr` Package

One of the faster packages that you may use to import your big data set into R could be the `readr` package, which allows you to read tabular text data, just like `read.table`.

Nevertheless, the `readr` package offers “a number of replacement functions that provide additional functionality and are much faster” (see [here](#)).

```
df <- read_table("<Path to your file>",  
                 col_names=TRUE)
```



POWERED BY DATACAMP WORKSPACE

Note that the `readr` package also offers the functions `read_csv()`, `read_csv2()`, `read_delim()`, `read_fwf()`, `read_tsv()` and many other functions that go faster than their original ones! Details can be found [here](#).

Tip: more information on this package can be found on [this GitHub page](#).

Some Remarks On Handling Big Data in R

For further tips on handling big data in R, you should probably take a look at [this StackOverflow discussion](#), which deals with packages but also with tips such as storing your data in binary formats and the usage of `saveRDS` / `readRDS` or the `rhdf5` package for the HDF5 format.

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

Make sure to also check out [this interesting post](#), which tests the load performance of some of the packages listed above!

Getting your Data into R with the `rio` Package

This “Swiss-army knife for data Input/Output” makes the input and output of data into R easier for you! You can in- or output data from almost any file format: when you install the `rio` package, you pull a lot of separate data-reading packages into one. If you then want to input or output data, you just need to remember two functions: `import()` and `export()`: `rio` will rely on the separate data-reading packages to infer the data structure from the file extension, to natively read web-based data sources and to set reasonable defaults for import and export.

In short, `rio` supports a broad set of commonly used file types for import and export.

Importing your files with `rio` happens in the following way:

```
library(rio)
data <- import("<Path to your file>")
```



POWERED BY DATACAMP WORKSPACE

If you want to see exactly which file formats are supported by `rio`, visit [this page](#).

TOPICS

On an Endpoint

R Programming Data Science

If you're interested in learning more about working with big data in R, make sure to check out the [How To Work With Quandl in R](#) and the “[Big Data Analysis With Revolution R Enterprise](#)” courses at DataCamp!

Learn more about R

Introduction to Importing Data in R

Beginner ⌚ 3 hr 👥 174.9K

In this course, you will learn to read CSV, XLS, and text files in R using tools like `readxl` and `data.table`.

II

L

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

Related



Building Your Data Science
Portfolio with DataCamp...

Justin Saddlemyer

[Radar Recap] Building an
Enterprise Data Strategy that...

Adel Nehme

[Radar Recap] Unleashing the
Power of Data Teams in 2023

Richie Cotton

[See More →](#)

Grow your data skills with DataCamp for Mobile

Make progress on the go with our mobile courses and daily 5-minute coding challenges.



LEARN

[Learn Python](#)

[Learn R](#)

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

Career Tracks

Skill Tracks

Courses

Data Science Roadmap

DATA COURSES

Upcoming Courses

Python Courses

R Courses

SQL Courses

Power BI Courses

Tableau Courses

Spreadsheets Courses

Data Analysis Courses

Data Visualization Courses

Machine Learning Courses

Data Engineering Courses

WORKSPACE

Get Started

Templates

Integrations

Documentation

CERTIFICATION

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept

RESOURCES

[Resource Center](#)

[Upcoming Events](#)

[Blog](#)

[Tutorials](#)

[Open Source](#)

[RDocumentation](#)

[Course Editor](#)

[Book a Demo with DataCamp for Business](#)

PLANS

[Pricing](#)

[For Business](#)

[For Universities](#)

[Discounts, Promos & Sales](#)

[DataCamp Donates](#)

SUPPORT

[Help Center](#)

[Become an Instructor](#)

[Become an Affiliate](#)

ABOUT

[About Us](#)

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept



[Privacy Policy](#) [Cookie Notice](#) [Do Not Sell My Personal Information](#) [Accessibility](#) [Security](#)
[Terms of Use](#)

© 2023 DataCamp, Inc. All Rights Reserved.

DataCamp and our partners use cookies and similar technologies to improve your learning experience, offer data science content relevant to your interests, improve the site and to show more relevant advertisements. You can change your mind at any time.

[Learn more & configure](#)

Accept